InsightFlow Analytics: Complete Build Guide

CausalFunnel Full Stack Engineer assignment · Tracker SDK, Express API, MongoDB, React dashboard

1. Plan at a glance

Build the core flow first, then polish. If time runs out, the core must still work end to end.

Priority	What	Notes
Must	Tracker: page_view, click, session_id, URL, timestamp, x/y	Vanilla JS, one script tag
Must	API: ingest, sessions list, session events, page click data	Node.js + Express
Must	MongoDB storage with indexes	Mongoose
Must	Dashboard: sessions + journey, heatmap with page selector	React
Must	Public GitHub repo + README	Setup, tech stack, assumptions
Should	Hosting, seed script, validation, duplicate protection, a few tests	Makes it feel finished
Later	Batching, auth, screenshot overlay, bot filtering	List under Future work in the README

Suggested time (about 25 working hours inside the 72-hour window):

- Day 1: backend, MongoDB, seed script, curl checks (6h); tracker and demo site (4h)
- Day 2: dashboard sessions, journey, heatmap (8h)
- Day 3: tests, deployment, README, final QA (6h)

2. Decisions that refine the earlier architecture

- No npm workspaces. Each app has its own package.json, which keeps Render and Vercel deployment simple.

- The tracker is bundled with esbuild into `apps/api/public/tracker.js` and committed. Express serves it at `/sdk/tracker.js`, so no separate CDN or cross-folder build is needed on Render.
- Ingestion CORS is open (no credentials). The Origin check happens inside the handler, because a CORS preflight cannot know the site_id. Dashboard routes are restricted to `DASHBOARD_ORIGIN`.
- The tracker posts `text/plain` using `fetch` with `keepalive: true`. This avoids a preflight request and still survives page navigation.
- Inserts are idempotent: a retried event_id returns success instead of an error.
- A `GET /api/sites` endpoint feeds the dashboard's site selector.
- A seed script creates the demo site and synthetic sessions, so an evaluator sees data immediately.
- Read APIs are unauthenticated because the data is synthetic. State this as a trade-off in the README.

3. Prerequisites (Fedora)

```
sudo dnf install -y nodejs npm git
node -v
podman run -d --name mongo -p 27017:27017 docker.io/library/mongo:7
```

Use Node 20 or newer. As an alternative to the container, create a free MongoDB Atlas M0 cluster and use its connection string as `MONGODB_URI`.

Express 5 (the current default) forwards errors from async handlers automatically. On Express 4 you would need a wrapper.

4. Repository setup

```
mkdir insightflow-analytics && cd insightflow-analytics
git init
mkdir -p
apps/api/src/{config,models,routes,controllers,services,middleware,validators
}
mkdir -p apps/api/{public,scripts,tests} apps/dashboard apps/demo-site
packages/tracker/src
echo node_modules > .gitignore
echo .env >> .gitignore
```

5. Backend (apps/api)

5.1 Install and configure

```
cd apps/api
npm init -y
npm pkg set type=module
npm pkg set scripts.dev='nodemon src/server.js'
npm pkg set scripts.start='node src/server.js'
npm pkg set scripts.seed='node scripts/seed.js'
npm pkg set scripts.test='vitest run'
npm i express mongoose cors dotenv zod helmet express-rate-limit
npm i -D nodemon vitest supertest
```

Create `apps/api/.env` (never commit it) and a copy named `.env.example` with placeholder values:

```
PORT=5000
NODE_ENV=development
MONGODB_URI=mongodb://127.0.0.1:27017/insightflow
DASHBOARD_ORIGIN=http://localhost:5173
DEMO_ORIGINS=http://localhost:5174
```

5.2 Database connection and models

src/config/database.js

```
import mongoose from 'mongoose';

export async function connectDB(uri) {
  await mongoose.connect(uri);
  console.log('MongoDB connected');
}
```

src/models/Site.js

```
import mongoose from 'mongoose';

const siteSchema = new mongoose.Schema(
  {
    _id: { type: String },
    name: { type: String, required: true },
    allowed_origins: { type: [String], default: [] },
  },
  { timestamps: { createdAt: 'created_at', updatedAt: false } }
);

export default mongoose.model('Site', siteSchema);
```

src/models/Event.js

```
import mongoose from 'mongoose';

const clickSchema = new mongoose.Schema(
  {
    x: Number,
    y: Number,
    viewport_width: Number,
    viewport_height: Number,
    document_width: Number,
    document_height: Number,
  },
  { _id: false }
);

const eventSchema = new mongoose.Schema({
  site_id: { type: String, required: true },
  event_id: { type: String, required: true },
  session_id: { type: String, required: true },
  event_type: { type: String, enum: ['page_view', 'click'], required: true },
```

```
  page_url: { type: String, required: true },
  timestamp: { type: Date, required: true },
  received_at: { type: Date, default: Date.now },
  click: clickSchema,
});

eventSchema.index({ site_id: 1, session_id: 1, timestamp: 1 });
eventSchema.index({ site_id: 1, page_url: 1, event_type: 1 });
eventSchema.index({ site_id: 1, event_id: 1 }, { unique: true });

export default mongoose.model('Event', eventSchema);
```

5.3 Validation

src/validators/event.schema.js

```
import { z } from 'zod';

const dim = z.number().int().positive().max(100000);

const clickSchema = z.object({
  x: z.number().min(0).max(100000),
  y: z.number().min(0).max(100000),
  viewport_width: dim,
  viewport_height: dim,
  document_width: dim,
  document_height: dim,
});

export const eventSchema = z
  .object({
    site_id: z.string().min(1).max(100),
    event_id: z.string().uuid(),
    session_id: z.string().min(8).max(100),
    event_type: z.enum(['page_view', 'click']),
    page_url: z.string().url().max(2048),
    timestamp: z.string().datetime(),
    click: clickSchema.optional(),
  })
```

```
  .refine((e) => e.event_type !== 'click' || e.click, {
    message: 'click data is required for click events',
    path: ['click'],
  });
```

5.4 Error handling

src/middleware/errorHandler.js

```
import { ZodError } from 'zod';

export function notFound(req, res) {
  res.status(404).json({ error: 'Not found' });
}

export function errorHandler(err, req, res, next) {
  if (err instanceof ZodError) {
    return res.status(400).json({
      error: 'Validation failed',
      details: err.issues.map((i) => ({ path: i.path.join('.'), message:
i.message })),
    });
  }
  if (err.status) return res.status(err.status).json({ error: err.message });
  console.error(err);
  res.status(500).json({ error: 'Internal server error' });
}
```

5.5 Event ingestion

src/controllers/events.controller.js

```
import Event from '../models/Event.js';
import Site from '../models/Site.js';
import { eventSchema } from '../validators/event.schema.js';

const httpError = (status, message) => Object.assign(new Error(message), {
status });

export async function ingestEvent(req, res) {
  let body;
  try {
```

```
    body = JSON.parse(req.body);
  } catch {
    throw httpError(400, 'Body must be valid JSON');
  }
  const data = eventSchema.parse(body);

  const site = await Site.findById(data.site_id).lean();
  if (!site) throw httpError(404, 'Unknown site_id');

  const origin = req.get('Origin');
  if (origin && !site.allowed_origins.includes(origin)) {
    throw httpError(403, 'Origin not allowed for this site');
  }

  const ts = new Date(data.timestamp);
  if (ts.getTime() > Date.now() + 5 * 60 * 1000) throw httpError(400,
'timestamp is in the future');

  try {
    await Event.create({ ...data, timestamp: ts });
  } catch (err) {
    if (err.code === 11000) return res.status(200).json({ ok: true,
duplicate: true });
    throw err;
  }
  res.status(202).json({ ok: true });
}
```

src/routes/events.routes.js

```
import { Router } from 'express';
import { ingestEvent } from '../controllers/events.controller.js';

const router = Router();
router.post('/', ingestEvent);
export default router;
```

5.6 Analytics queries

src/services/analytics.service.js

```
import Event from '../models/Event.js';

const countIf = (type) => ({ $sum: { $cond: [{ $eq: ['$event_type', type] },
1, 0] } });

export async function listSessions(siteId, page, limit) {
  const [sessions, totalRow] = await Promise.all([
    Event.aggregate([
      { $match: { site_id: siteId } },
      {
        $group: {
          _id: '$session_id',
          event_count: { $sum: 1 },
          page_views: countIf('page_view'),
          clicks: countIf('click'),
          started_at: { $min: '$timestamp' },
          last_seen_at: { $max: '$timestamp' },
        },
      },
      { $sort: { last_seen_at: -1 } },
      { $skip: (page - 1) * limit },
      { $limit: limit },
      {
        $project: {
          _id: 0,
          session_id: '$_id',
          event_count: 1,
          page_views: 1,
          clicks: 1,
          started_at: 1,
          last_seen_at: 1,
        },
      },
    ]),
    Event.aggregate([
      { $match: { site_id: siteId } },
      { $group: { _id: '$session_id' } },
      { $count: 'total' },
    ]),
  ]);
  return { sessions, page, limit, total: totalRow[0]?.total ?? 0 };
}
```

```
export function getSessionEvents(siteId, sessionId) {
  return Event.find({ site_id: siteId, session_id: sessionId })
    .sort({ timestamp: 1, _id: 1 })
    .select('-_id event_type page_url timestamp click.x click.y')
    .lean();
}

export function listPages(siteId) {
  return Event.aggregate([
    { $match: { site_id: siteId } },
    { $group: { _id: '$page_url', views: countIf('page_view'), clicks:
countIf('click') } },
    { $sort: { clicks: -1 } },
    { $project: { _id: 0, page_url: '$_id', views: 1, clicks: 1 } },
  ]);
}

export async function getHeatmap(siteId, pageUrl, limit = 5000) {
  const filter = { site_id: siteId, page_url: pageUrl, event_type: 'click' };
  const [total, rows] = await Promise.all([
    Event.countDocuments(filter),
    Event.find(filter).sort({ timestamp: -1 }).limit(limit).select('-_id
click').lean(),
  ]);
  return { page_url: pageUrl, total_clicks: total, clicks: rows.map((r) =>
r.click) };
}
```

src/controllers/analytics.controller.js

```
import { z } from 'zod';
import Site from '../models/Site.js';
import * as svc from '../services/analytics.service.js';

const notFoundError = (msg) => Object.assign(new Error(msg), { status: 404
});

const pageQuery = z.object({
  page: z.coerce.number().int().min(1).default(1),
  limit: z.coerce.number().int().min(1).max(100).default(20),
});

async function requireSite(siteId) {
```

```
  if (!(await Site.exists({ _id: siteId }))) throw notFoundError('Site not
found');
}

export const sites = async (req, res) => {
  res.json({ sites: await Site.find().select('name').lean() });
};

export const sessions = async (req, res) => {
  await requireSite(req.params.siteId);
  const { page, limit } = pageQuery.parse(req.query);
  res.json(await svc.listSessions(req.params.siteId, page, limit));
};

export const sessionEvents = async (req, res) => {
  const { siteId, sessionId } = req.params;
  const events = await svc.getSessionEvents(siteId, sessionId);
  if (!events.length) throw notFoundError('Session not found');
  res.json({ session_id: sessionId, events });
};

export const pages = async (req, res) => {
  await requireSite(req.params.siteId);
  res.json({ pages: await svc.listPages(req.params.siteId) });
};

export const heatmap = async (req, res) => {
  await requireSite(req.params.siteId);
  const { url } = z.object({ url: z.string().url() }).parse(req.query);
  res.json(await svc.getHeatmap(req.params.siteId, url));
};
```

src/routes/analytics.routes.js

```
import { Router } from 'express';
import * as c from '../controllers/analytics.controller.js';

const router = Router();
router.get('/sites', c.sites);
router.get('/sites/:siteId/sessions', c.sessions);
router.get('/sites/:siteId/sessions/:sessionId/events', c.sessionEvents);
router.get('/sites/:siteId/pages', c.pages);
router.get('/sites/:siteId/heatmap', c.heatmap);
export default router;
```

5.7 App and server

src/app.js

```
import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import rateLimit from 'express-rate-limit';
import path from 'node:path';
import { fileURLToPath } from 'node:url';
import eventsRoutes from './routes/events.routes.js';
import analyticsRoutes from './routes/analytics.routes.js';
import { notFound, errorHandler } from './middleware/errorHandler.js';

const publicDir = path.join(path.dirname(fileURLToPath(import.meta.url)),
'..', 'public');

export function createApp() {
  const app = express();
  app.set('trust proxy', 1);

  // The tracker script is loaded from other origins, so CORP must be cross-
origin.
  app.use(helmet({ crossOriginResourcePolicy: { policy: 'cross-origin' } }));
  app.use('/sdk', express.static(publicDir, { maxAge: '5m' }));
  app.get('/api/health', (req, res) => res.json({ status: 'ok' }));

  // Public ingestion: any origin may POST; the handler checks the site's
allowed origins.
  app.use(
    '/api/events',
```

```
    cors({ origin: '*', methods: ['POST', 'OPTIONS'] }),
    rateLimit({ windowMs: 60_000, limit: 300 }),
    express.text({ type: () => true, limit: '10kb' }),
    eventsRoutes
  );

  // Dashboard reads: restricted to the dashboard origin(s).
  const dashboardOrigins = (process.env.DASHBOARD_ORIGIN ||
'').split(',').filter(Boolean);
  app.use('/api', cors({ origin: dashboardOrigins }), analyticsRoutes);

  app.use(notFound);
  app.use(errorHandler);
  return app;
}
```

src/server.js

```
import 'dotenv/config';
import { connectDB } from './config/database.js';
import { createApp } from './app.js';

await connectDB(process.env.MONGODB_URI);
const port = process.env.PORT || 5000;
createApp().listen(port, () => console.log(`API listening on ${port}`));
```

5.8 Seed script

scripts/seed.js creates the demo site and 30 synthetic sessions with click hotspots, so the heatmap looks meaningful right away.

```
import 'dotenv/config';
import crypto from 'node:crypto';
import mongoose from 'mongoose';
import Site from '../src/models/Site.js';
import Event from '../src/models/Event.js';

const SITE_ID = 'demo_store';
const origins = (process.env.DEMO_ORIGINS ||
'http://localhost:5174').split(',');
const base = origins[0];
const paths = ['/', '/products.html', '/cart.html'];
const hotspots = [[0.5, 0.12], [0.3, 0.4], [0.7, 0.62]];
```

```
const rand = (min, max) => Math.round(min + Math.random() * (max - min));

const makeEvent = (session_id, event_type, page_url, t, click) => ({
  site_id: SITE_ID,
  event_id: crypto.randomUUID(),
  session_id,
  event_type,
  page_url,
  timestamp: new Date(t),
  ...(click && { click }),
});

await mongoose.connect(process.env.MONGODB_URI);
await Site.findByIdAndUpdate(
  SITE_ID,
  { name: 'Demo Store', allowed_origins: origins },
  { upsert: true }
);
if (process.argv.includes('--reset')) await Event.deleteMany({ site_id:
SITE_ID });
await Event.init();

const events = [];
for (let s = 0; s < 30; s++) {
  const session_id = crypto.randomUUID();
  const desktop = Math.random() < 0.7;
  const vw = desktop ? 1440 : 390;
  const vh = desktop ? 900 : 844;
  const docH = desktop ? 2400 : 4200;
  let t = Date.now() - rand(1, 72) * 3600_000;

  for (const p of paths.slice(0, rand(1, 3))) {
    const url = base + p;
    events.push(makeEvent(session_id, 'page_view', url, t));
    const clicks = rand(2, 8);
    for (let c = 0; c < clicks; c++) {
      t += rand(2, 20) * 1000;
      const [hx, hy] = hotspots[rand(0, hotspots.length - 1)];
      const jitter = () => (Math.random() - 0.5) * 0.08;
      events.push(
        makeEvent(session_id, 'click', url, t, {
          x: Math.round(vw * (hx + jitter())),
          y: Math.round(docH * (hy + jitter())),
```

```
          viewport_width: vw,
          viewport_height: vh,
          document_width: vw,
          document_height: docH,
        })
      );
    }
    t += 5000;
  }
}

await Event.insertMany(events);
console.log(`Seeded ${events.length} events`);
await mongoose.disconnect();
```

Run `npm run seed -- --reset` any time to regenerate the data.

5.9 Verify the API before building anything else

```
npm run seed
npm run dev
curl localhost:5000/api/health
curl 'localhost:5000/api/sites/demo_store/sessions?page=1&limit=5'
curl -i -X POST localhost:5000/api/events -H 'Content-Type: text/plain' -d
'{"site_id":"demo_store","event_id":"550e8400-e29b-41d4-a716-
446655440001","session_id":"manual-test-
1","event_type":"page_view","page_url":"http://localhost:5174/","timestamp":"
2026-10-12T10:00:00.000Z"}'
```

The POST should return 202. Run it a second time and it should return 200 with `duplicate: true`. Then fetch the sessions list and confirm `manual-test-1` appears.

6. Tracker (packages/tracker)

```
cd packages/tracker
npm init -y
npm i -D esbuild
npm pkg set scripts.build='esbuild src/tracker.js --bundle --minify --
format=iife --target=es2018 --outfile=../../apps/api/public/tracker.js'
npm pkg set scripts.watch='npm run build -- --watch'
```

src/tracker.js (you can split it into the config, session, event-builder and transport modules later; esbuild will bundle them).

```
(function () {
  try {
    const script = document.currentScript; // must be read synchronously
    if (!script || window.__insightflow) return;
    const siteId = script.getAttribute('data-site-id');
    if (!siteId || location.origin === 'null') return;
    window.__insightflow = true;
    const apiUrl = script.getAttribute('data-api-url') || new
URL('/api/events', script.src).href;

    const KEY = 'insightflow:' + siteId + ':session';
    const TIMEOUT = 30 * 60 * 1000;
    let memory = null; // fallback when localStorage is blocked

    const uuid = () =>
      crypto.randomUUID
        ? crypto.randomUUID()
        : 'xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx'.replace(/[xy]/g, (c) => {
            const r = (Math.random() * 16) | 0;
            return (c === 'x' ? r : (r & 3) | 8).toString(16);
          });

    function sessionId() {
      const now = Date.now();
      let s = null;
      try {
        s = JSON.parse(localStorage.getItem(KEY));
      } catch (e) {}
      s = s || memory;
      if (!s || !s.id || now - s.last > TIMEOUT) s = { id: uuid(), last: now
};
      s.last = now; // sliding 30-minute window
      memory = s;
      try {
        localStorage.setItem(KEY, JSON.stringify(s));
      } catch (e) {}
      return s.id;
    }

    const pageUrl = () => location.origin + location.pathname; // no query or
```

```
hash

    function send(evt) {
      try {
        fetch(apiUrl, {
          method: 'POST',
          headers: { 'Content-Type': 'text/plain;charset=UTF-8' },
          body: JSON.stringify(evt),
          keepalive: true,
          credentials: 'omit',
          mode: 'cors',
        }).catch(() => {});
      } catch (e) {}
    }

    function track(type, extra) {
      send(
        Object.assign(
          {
            site_id: siteId,
            event_id: uuid(),
            session_id: sessionId(),
            event_type: type,
            page_url: pageUrl(),
            timestamp: new Date().toISOString(),
          },
          extra
        )
      );
    }

    let lastPath = location.pathname;
    track('page_view');

    // Capture phase, so clicks are recorded even if the app stops
propagation.
    document.addEventListener(
      'click',
      (e) => {
        const de = document.documentElement;
        const b = document.body;
        track('click', {
          click: {
```

```
            x: Math.round(e.pageX),
            y: Math.round(e.pageY),
            viewport_width: window.innerWidth,
            viewport_height: window.innerHeight,
            document_width: Math.max(de.scrollWidth, b ? b.scrollWidth : 0),
            document_height: Math.max(de.scrollHeight, b ? b.scrollHeight :
0),
          },
        });
      },
      true
    );

    // Optional: single-page apps. pushState does not fire popstate, so patch
it.
    const onRoute = () => {
      if (location.pathname !== lastPath) {
        lastPath = location.pathname;
        track('page_view');
      }
    };
    ['pushState', 'replaceState'].forEach((m) => {
      const orig = history[m];
      history[m] = function () {
        const r = orig.apply(this, arguments);
        setTimeout(onRoute, 0);
        return r;
      };
    });
    window.addEventListener('popstate', onRoute);
  } catch (e) {
    // Analytics must never break the host page.
  }
})();
```

Run `npm run build`, then commit `apps/api/public/tracker.js`.

7. Demo site (apps/demo-site)

Create `index.html`, `products.html` and `cart.html`. They are identical apart from the title and heading. The `.tall` block makes the page long enough to test scroll-aware click positions.

```
<!doctype html>
<html lang='en'>
<head>
  <meta charset='utf-8'>
  <meta name='viewport' content='width=device-width, initial-scale=1'>
  <title>Demo Store - Home</title>
  <style>
    body { font-family: system-ui, sans-serif; margin: 0; }
    nav { display: flex; gap: 16px; padding: 16px 24px; background: #111; }
    nav a { color: #fff; text-decoration: none; }
    main { max-width: 900px; margin: 0 auto; padding: 24px; }
    button { padding: 10px 18px; margin: 8px 8px 8px 0; cursor: pointer; }
    .tall { height: 1600px; background: linear-gradient(#fff, #e6f0ff); }
  </style>
</head>
<body>
  <nav>
    <a href='/'>Home</a>
    <a href='/products.html'>Products</a>
    <a href='/cart.html'>Cart</a>
  </nav>
  <main>
    <h1>Welcome to Demo Store</h1>
    <button>Shop now</button><button>Learn more</button>
    <div class='tall'><button>Button far down the page</button></div>
  </main>
  <script async src='http://localhost:5000/sdk/tracker.js' data-site-
id='demo_store' data-api-url='http://localhost:5000/api/events'></script>
</body>
</html>
```

Serve it on port 5174 (this keeps the `.html` paths, which match the seed data):

```
python3 -m http.server 5174 --directory apps/demo-site
```

Open http://localhost:5174, click around, switch pages, then check the API: new sessions and events should appear.

8. Dashboard (apps/dashboard)

8.1 Setup

```
cd apps/dashboard
npm create vite@latest . -- --template react
npm install
npm i tailwindcss @tailwindcss/vite
```

Delete `src/App.css` and the default assets. Create `.env` with `VITE_API_URL=http://localhost:5000`.

vite.config.js

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({ plugins: [react(), tailwindcss()] });
```

src/index.css (replace the contents)

```
@import 'tailwindcss';
```

Make sure `src/main.jsx` imports `./index.css`.

8.2 API client and data hook

src/services/api.js

```
const BASE = import.meta.env.VITE_API_URL || 'http://localhost:5000';

async function get(path, params = {}) {
  const url = new URL(BASE + path);
  Object.entries(params).forEach(([k, v]) => url.searchParams.set(k, v));
  const res = await fetch(url);
  if (!res.ok) {
    const body = await res.json().catch(() => ({}));
    throw new Error(body.error || `Request failed (${res.status})`);
  }
  return res.json();
}
```

```
export const api = {
  sites: () => get('/api/sites'),
  sessions: (siteId, params) => get(`/api/sites/${siteId}/sessions`, params),
  sessionEvents: (siteId, id) =>
get(`/api/sites/${siteId}/sessions/${id}/events`),
  pages: (siteId) => get(`/api/sites/${siteId}/pages`),
  heatmap: (siteId, url) => get(`/api/sites/${siteId}/heatmap`, { url }),
};
```

src/hooks/useAsync.js gives every view consistent loading, error and empty handling.

```
import { useEffect, useState } from 'react';

export function useAsync(fn, deps) {
  const [state, setState] = useState({ loading: true, data: null, error: null
});
  useEffect(() => {
    let live = true;
    setState((s) => ({ ...s, loading: true, error: null }));
    fn()
      .then((data) => live && setState({ loading: false, data, error: null
}))
      .catch((e) => live && setState({ loading: false, data: null, error:
e.message }));
    return () => {
      live = false;
    };
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, deps);
  return state;
}
```

8.3 App shell

src/App.jsx

```
import { useState } from 'react';
import { api } from './services/api.js';
import { useAsync } from './hooks/useAsync.js';
import Sessions from './pages/Sessions.jsx';
import Heatmap from './pages/Heatmap.jsx';

export default function App() {
```

```
  const [view, setView] = useState('sessions');
  const [siteId, setSiteId] = useState('');
  const { data, error, loading } = useAsync(() => api.sites(), []);
  const sites = data?.sites ?? [];
  const active = siteId || sites[0]?._id;

  return (
    <div className='min-h-screen bg-slate-50 text-slate-900'>
      <header className='flex flex-wrap items-center gap-4 border-b bg-white
px-6 py-3'>
        <h1 className='text-lg font-semibold'>InsightFlow</h1>
        <select className='rounded border px-2 py-1' value={active || ''}
onChange={(e) => setSiteId(e.target.value)}>
          {sites.map((s) => (
            <option key={s._id} value={s._id}>{s.name}</option>
          ))}
        </select>
        {['sessions', 'heatmap'].map((v) => (
          <button
            key={v}
            onClick={() => setView(v)}
            className={`rounded px-3 py-1 capitalize ${view === v ? 'bg-
slate-900 text-white' : 'hover:bg-slate-100'}`}
          >
            {v}
          </button>
        ))}
      </header>
      <main className='mx-auto max-w-6xl p-6'>
        {loading && <p>Loading sites...</p>}
        {error && <p className='text-red-600'>{error}</p>}
        {!loading && !error && !active && <p>No sites found. Run the seed
script.</p>}
        {active && (view === 'sessions' ? <Sessions key={active} siteId=
{active} /> : <Heatmap key={active} siteId={active} />)}
      </main>
    </div>
  );
}
```

8.4 Sessions view

src/pages/Sessions.jsx

```
import { useState } from 'react';
import { api } from '../services/api.js';
import { useAsync } from '../hooks/useAsync.js';
import EventTimeline from '../components/EventTimeline.jsx';

const PAGE_SIZE = 10;

export default function Sessions({ siteId }) {
  const [page, setPage] = useState(1);
  const [selected, setSelected] = useState(null);
  const list = useAsync(() => api.sessions(siteId, { page, limit: PAGE_SIZE
}), [siteId, page]);
  const journey = useAsync(
    () => (selected ? api.sessionEvents(siteId, selected) :
Promise.resolve(null)),
    [siteId, selected]
  );
  const totalPages = Math.max(1, Math.ceil((list.data?.total ?? 0) /
PAGE_SIZE));

  return (
    <div className='grid gap-6 lg:grid-cols-2'>
      <section>
        <h2 className='mb-3 text-xl font-semibold'>Visitor sessions</h2>
        {list.loading && <p>Loading...</p>}
        {list.error && <p className='text-red-600'>{list.error}</p>}
        {list.data?.sessions.length === 0 && <p>No sessions yet.</p>}
        {list.data?.sessions.length > 0 && (
          <table className='w-full overflow-hidden rounded border bg-white
text-sm'>
            <thead className='bg-slate-100 text-left'>
              <tr><th className='p-2'>Session</th><th>Events</th><th>Last
seen</th></tr>
            </thead>
            <tbody>
              {list.data.sessions.map((s) => (
                <tr
                  key={s.session_id}
                  onClick={() => setSelected(s.session_id)}
                  className={`cursor-pointer border-t hover:bg-blue-50
${selected === s.session_id ? 'bg-blue-50' : ''}`}
                >
                  <td className='p-2 font-mono'>{s.session_id.slice(0, 8)}
```

```
</td>
                  <td>{s.event_count}</td>
                  <td>{new Date(s.last_seen_at).toLocaleString()}</td>
                </tr>
              ))}
            </tbody>
          </table>
        )}
        <div className='mt-3 flex items-center gap-3 text-sm'>
          <button disabled={page <= 1} onClick={() => setPage(page - 1)}
className='rounded border px-3 py-1 disabled:opacity-40'>Prev</button>
          <span>Page {page} of {totalPages}</span>
          <button disabled={page >= totalPages} onClick={() => setPage(page +
1)} className='rounded border px-3 py-1 disabled:opacity-40'>Next</button>
        </div>
      </section>

      <section>
        <h2 className='mb-3 text-xl font-semibold'>User journey</h2>
        {!selected && <p className='text-slate-500'>Select a session to see
its events.</p>}
        {selected && journey.loading && <p>Loading...</p>}
        {journey.error && <p className='text-red-600'>{journey.error}</p>}
        {journey.data && <EventTimeline events={journey.data.events} />}
      </section>
    </div>
  );
}
```

src/components/EventTimeline.jsx

```
export default function EventTimeline({ events }) {
  return (
    <ol className='space-y-2'>
      {events.map((e, i) => (
        <li key={i} className='rounded border bg-white p-3 text-sm'>
          <span className='font-medium'>{i + 1}. {e.event_type === 'click' ?
'Click' : 'Page view'}</span>
          <span className='ml-2 text-slate-500'>{new
Date(e.timestamp).toLocaleTimeString()}</span>
          <div className='font-mono text-xs text-slate-600'>
            {new URL(e.page_url).pathname}
            {e.click && ` · (${e.click.x}, ${e.click.y})`}
          </div>
        </li>
      ))}
    </ol>
  );
}
```

8.5 Heatmap view

src/pages/Heatmap.jsx

```
import { useEffect, useState } from 'react';
import { api } from '../services/api.js';
import { useAsync } from '../hooks/useAsync.js';
import ClickHeatmap from '../components/ClickHeatmap.jsx';

export default function Heatmap({ siteId }) {
  const [url, setUrl] = useState('');
  const [device, setDevice] = useState('all');
  const [mode, setMode] = useState('heat');
  const pages = useAsync(() => api.pages(siteId), [siteId]);
  const map = useAsync(() => (url ? api.heatmap(siteId, url) :
Promise.resolve(null)), [siteId, url]);

  useEffect(() => {
    if (!url && pages.data?.pages[0]) setUrl(pages.data.pages[0].page_url);
  }, [pages.data, url]);

  const clicks = (map.data?.clicks ?? []).filter((c) =>
    device === 'all' ? true : device === 'mobile' ? c.viewport_width < 768 :
```

```
c.viewport_width >= 768
  );

  return (
    <div>
      <h2 className='mb-3 text-xl font-semibold'>Click heatmap</h2>
      <div className='mb-4 flex flex-wrap gap-3'>
        <select value={url} onChange={(e) => setUrl(e.target.value)}
className='rounded border px-2 py-1'>
          {pages.data?.pages.map((p) => (
            <option key={p.page_url} value={p.page_url}>{p.page_url}
({p.clicks} clicks)</option>
          ))}
        </select>
        <select value={device} onChange={(e) => setDevice(e.target.value)}
className='rounded border px-2 py-1'>
          <option value='all'>All devices</option>
          <option value='desktop'>Desktop</option>
          <option value='mobile'>Mobile</option>
        </select>
        <select value={mode} onChange={(e) => setMode(e.target.value)}
className='rounded border px-2 py-1'>
          <option value='heat'>Heat</option>
          <option value='dots'>Dots</option>
        </select>
      </div>
      {pages.error && <p className='text-red-600'>{pages.error}</p>}
      {pages.data?.pages.length === 0 && <p>No pages tracked yet.</p>}
      {map.loading && url && <p>Loading...</p>}
      {map.error && <p className='text-red-600'>{map.error}</p>}
      {map.data && clicks.length === 0 && <p>No clicks recorded for this
selection.</p>}
      {clicks.length > 0 && <ClickHeatmap clicks={clicks} mode={mode} />}
      <p className='mt-2 text-sm text-slate-500'>
        {clicks.length} clicks shown. Each click is scaled by the size of the
page it happened on, so the map is approximate across different layouts.
      </p>
    </div>
  );
}
```

src/components/ClickHeatmap.jsx draws on a canvas. Overlapping soft gradients add up, so busy areas get hotter.

```
import { useEffect, useRef } from 'react';

const W = 800;
const median = (arr) => [...arr].sort((a, b) => a - b)[Math.floor(arr.length
/ 2)];

export default function ClickHeatmap({ clicks, mode }) {
  const ref = useRef(null);
  const ratio = median(clicks.map((c) => c.document_height)) /
median(clicks.map((c) => c.document_width));
  const H = Math.min(4000, Math.round(W * ratio));

  useEffect(() => {
    const ctx = ref.current.getContext('2d');
    ctx.clearRect(0, 0, W, H);
    ctx.fillStyle = '#fff';
    ctx.fillRect(0, 0, W, H);
    ctx.strokeStyle = '#e2e8f0';
    for (let i = 1; i < 10; i++) {
      ctx.beginPath(); ctx.moveTo((W * i) / 10, 0); ctx.lineTo((W * i) / 10,
H); ctx.stroke();
      ctx.beginPath(); ctx.moveTo(0, (H * i) / 10); ctx.lineTo(W, (H * i) /
10); ctx.stroke();
    }
    for (const c of clicks) {
      const x = (c.x / c.document_width) * W;
      const y = (c.y / c.document_height) * H;
      if (mode === 'dots') {
        ctx.fillStyle = 'rgba(220,38,38,0.7)';
        ctx.beginPath(); ctx.arc(x, y, 5, 0, Math.PI * 2); ctx.fill();
      } else {
        const g = ctx.createRadialGradient(x, y, 0, x, y, 28);
        g.addColorStop(0, 'rgba(239,68,68,0.35)');
        g.addColorStop(1, 'rgba(239,68,68,0)');
        ctx.fillStyle = g;
        ctx.fillRect(x - 28, y - 28, 56, 56);
      }
    }
  }, [clicks, mode, H]);
```

```
  return <canvas ref={ref} width={W} height={H} className='h-auto max-w-full
rounded border bg-white' />;
}
```

Run `npm run dev` and open http://localhost:5173. You should see the seeded sessions, a journey on click, and a heatmap with three hot spots.

9. Automated tests (apps/api)

Start MongoDB first. The tests use a separate database (`insightflow_test`) and drop it afterwards.

tests/api.test.js

```
import { beforeAll, afterAll, describe, expect, it } from 'vitest';
import request from 'supertest';
import mongoose from 'mongoose';
import { createApp } from '../src/app.js';
import Event from '../src/models/Event.js';
import Site from '../src/models/Site.js';

const app = createApp();
const click = { x: 10, y: 20, viewport_width: 1000, viewport_height: 800,
document_width: 1000, document_height: 2000 };
const ev = (over = {}) => ({
  site_id: 't_site',
  event_id: crypto.randomUUID(),
  session_id: 'session-test-1',
  event_type: 'page_view',
  page_url: 'http://localhost:5174/',
  timestamp: new Date().toISOString(),
  ...over,
});
const post = (body) =>
  request(app).post('/api/events').set('Content-Type',
'text/plain').send(JSON.stringify(body));

beforeAll(async () => {
  await mongoose.connect(process.env.MONGODB_TEST_URI ||
'mongodb://127.0.0.1:27017/insightflow_test');
  await Event.init();
```

```
  await Event.deleteMany({});
  await Site.deleteMany({});
  await Site.create({ _id: 't_site', name: 'Test', allowed_origins:
['http://localhost:5174'] });
});

afterAll(async () => {
  await mongoose.connection.dropDatabase();
  await mongoose.disconnect();
});

describe('POST /api/events', () => {
  it('accepts a valid page_view', async () => {
    expect((await post(ev())).status).toBe(202);
  });

  it('treats a repeated event_id as a duplicate, not an error', async () => {
    const body = ev();
    await post(body);
    const res = await post(body);
    expect(res.status).toBe(200);
    expect(res.body.duplicate).toBe(true);
  });

  it('rejects a click without coordinates', async () => {
    expect((await post(ev({ event_type: 'click' }))).status).toBe(400);
  });

  it('rejects an unknown site', async () => {
    expect((await post(ev({ site_id: 'nope' }))).status).toBe(404);
  });

  it('rejects an origin that is not allowed', async () => {
    const res = await post(ev()).set('Origin', 'http://evil.example');
    expect(res.status).toBe(403);
  });
});

describe('analytics endpoints', () => {
  it('lists sessions with counts and returns ordered events', async () => {
    const sid = 'session-order-1';
    const t0 = Date.now();
    await post(ev({ session_id: sid, timestamp: new Date(t0).toISOString()
```

```
}));
    await post(ev({ session_id: sid, event_type: 'click', click, timestamp:
new Date(t0 + 1000).toISOString() }));

    const list = await request(app).get('/api/sites/t_site/sessions');
    expect(list.body.sessions.find((s) => s.session_id ===
sid).event_count).toBe(2);

    const detail = await
request(app).get(`/api/sites/t_site/sessions/${sid}/events`);
    expect(detail.body.events.map((e) => e.event_type)).toEqual(['page_view',
'click']);
  });

  it('returns click data for a page', async () => {
    const res = await request(app).get('/api/sites/t_site/heatmap').query({
url: 'http://localhost:5174/' });
    expect(res.status).toBe(200);
    expect(res.body.clicks.length).toBeGreaterThan(0);
  });
});
```

Run them with `npm test`. Mention in the README that tracker behaviour is verified manually using the checklist in section 12.

10. Deployment

Deploy in this order, because each step needs a URL from the previous one.

1. **MongoDB Atlas.** Create a free M0 cluster and a database user. Under Network Access allow `0.0.0.0/0` (Render's free tier has no fixed IP). Copy the connection string and add the database name, e.g. `...mongodb.net/insightflow`.
2. **Seed production data.** From `apps/api` on your machine: `MONGODB_URI='<atlas uri>' DEMO_ORIGINS='https://<your-demo>.vercel.app,http://localhost:5174' npm run seed`. You can re-run this later once the demo URL is known.
3. **Render (API).** New Web Service from your GitHub repo. Root Directory `apps/api`, Build Command `npm install`, Start Command `npm start`, Health Check Path `/api/health`. Environment: `MONGODB_URI`, `NODE_ENV=production`, `NODE_VERSION=20`, and a temporary `DASHBOARD_ORIGIN`. Confirm that `https://<api>.onrender.com/sdk/tracker.js` loads.

4. **Vercel (dashboard).** Import the repo with Root Directory `apps/dashboard`, framework Vite, and the environment variable `VITE_API_URL=https://<api>.onrender.com`.
5. **Vercel (demo site).** A second project with Root Directory `apps/demo-site`. In all three HTML files change the script tag to `src='https://<api>.onrender.com/sdk/tracker.js'` and `data-api-url='https://<api>.onrender.com/api/events'`.
6. **Connect them.** Set Render's `DASHBOARD_ORIGIN` to the dashboard's Vercel URL (comma-separate it with `http://localhost:5173` if you want local access too). Re-run the seed with `DEMO_ORIGINS` set to the demo site's Vercel URL so its Origin is allowed.
7. **Test the hosted flow.** Visit the hosted demo, click around, and confirm the new session appears on the hosted dashboard.

Notes for the README: Render's free tier sleeps when idle, so the first request can take around a minute. The tracker fails silently, so very early clicks after a cold start may be lost. Mention this as a known limitation.

11. README outline

The assignment explicitly grades setup steps, tech stack, and assumptions or trade-offs. Cover these sections:

- **Overview and live links:** one paragraph, the hosted demo, the hosted dashboard, and the API URL.
- **Features:** tracker, sessions and journey, heatmap, validation, idempotent ingestion, seed data.
- **Architecture:** a short diagram such as: Website + tracker.js sends POST /api/events to the Express API, which writes to MongoDB; the React dashboard sends GET requests to the same API.
- **Tech stack:** a table with each technology and why it was chosen.
- **Setup:** prerequisites, MongoDB via container or Atlas, `cp .env.example .env`, `npm install` and `npm run seed` and `npm run dev` in `apps/api`, `npm run dev` in `apps/dashboard`, the `python3 -m http.server` command for the demo, and how to rebuild the tracker.
- **Embedding snippet:** the script tag with `data-site-id` and `data-api-url`.
- **API reference:** a table of the six endpoints, parameters and example responses.

- **Data model:** the `sites` and `events` documents and the three indexes, with one sentence on why each exists.
- **Assumptions and trade-offs:** see the list below.
- **Testing:** `npm test`, plus the manual checklist.
- **Future work:** batching, authentication, bot filtering, screenshot overlays, retention policy.

Assumptions and trade-offs to state honestly:

- A session is one browser on one site with at most 30 minutes of inactivity, stored in localStorage (with an in-memory fallback). This is a design choice, not part of the assignment.
- Page URLs are stored as origin plus pathname. Query strings and fragments are dropped to avoid capturing sensitive data, which means pages that differ only by query string are merged.
- The site ID is public. The Origin check stops casual misuse from other websites, but it is not authentication, because non-browser clients can forge headers.
- Read endpoints are unauthenticated because the demo uses synthetic data. A real deployment would add authentication before collecting visitor data.
- Events are sent one by one with `fetch` and `keepalive`, with no retry queue. This keeps the tracker small but can lose events when the network or API is down.
- The heatmap scales clicks by document size. Different responsive layouts can move content, so it is approximate, and desktop and mobile can be filtered separately.
- Sessions are computed from events with MongoDB aggregation instead of a separate collection. This is simple and consistent, but it will need a materialised sessions collection at large scale.
- No consent banner or bot filtering is included.

12. QA checklist and troubleshooting

Manual checklist:

- Fresh browser profile: the first visit creates a session_id in localStorage and sends one page_view.
- Reload: the same session_id is kept.
- Clicks near the top and far down the long page are stored with different y values.
- Navigating to another page keeps the session and records a new page_view.

- Editing the stored session timestamp to more than 30 minutes ago creates a new session on the next event.
- Stop the API and click around: the page keeps working with no visible errors.
- Blocked localStorage (private mode or site settings) still tracks within the page's lifetime.
- Sessions list counts match the number of rows in MongoDB for that session.
- The journey order matches what you did.
- The heatmap dots land where you clicked, and the desktop and mobile filters change the output.
- Empty, loading and error states all show something sensible.
- Hosted demo to hosted dashboard works in Chrome and Firefox.

Common problems:

Symptom	Likely cause and fix
tracker.js fails with a blocked-by-response error	Helmet's Cross-Origin-Resource-Policy header. Keep the `cross-origin` setting in app.js.
API returns 403 Origin not allowed	The demo's origin is not in `allowed_origins`. Re-run the seed with `DEMO_ORIGINS`. Origins include the port and have no trailing slash.
Console shows a CORS error from the dashboard	`DASHBOARD_ORIGIN` does not exactly match the dashboard URL.
Heatmap page list is empty	No click events yet, or the page URL in the data differs (trailing slash, http vs https).
Events appear for http but not https demo	Mixed content: an https page cannot call an http API. Use the https API URL.
`crypto.randomUUID` is not a function	Page served over plain http on a non-localhost host. The tracker has a fallback, so rebuild if you changed it.
Render service returns an error right after idle	Free tier cold start. Wait and retry.
Seed fails with a duplicate key error	Re-run with `--reset`, or check that you did not change the unique index.

13. Submission checklist and talking points

Before you submit:

- Public GitHub repo with a clear commit history (small commits per feature).
- `.env` is not committed and `.env.example` is.
- `apps/api/public/tracker.js` is committed and matches the source.
- A fresh clone works by following only the README.
- Hosted links work in a private window.
- No leftover boilerplate or unused files from the Vite template.
- `npm test` passes.

Be ready to explain these choices in a review, since they show engineering judgement:

- Why a vanilla JavaScript tracker, and how it avoids breaking host pages.
- Why the session lives in localStorage and what the 30-minute rule does.
- Why click coordinates are document-based, not viewport-based.
- Why the Origin check is not authentication, and what you would add next.
- Why `text/plain` with `keepalive` fetch avoids a CORS preflight.
- Why a unique index on `site_id` and `event_id` makes retries safe.
- How compound indexes support the sessions and heatmap queries, and how you would scale sessions further.
- What you deliberately left out for the time window and why.